



Devoir – Amélioration UI de l'application « Profil développeur »

Vous avez déjà créé une première application **Profil développeur**.

Cette première version permettait d'afficher une fiche simple avec des composables réutilisables, des paramètres, des *modifiers*, des *callbacks*, des *previews* et quelques bonnes pratiques de base.

À ce stade, vous devez faire évoluer cette application pour la transformer en une **petite interface de profil plus complète et plus professionnelle**.

Attention ! N'écrasez pas l'ancien dépôt Git du premier devoir, car il reste utilisé pour la notation : créez un **nouveau dépôt public** pour cette version, en repartant éventuellement d'un clone ou d'un fork de votre ancien projet.

L'objectif est de réutiliser les notions vues dans ce chapitre **Compose UI** :

- composants fondamentaux ;
- layouts Compose ;
- listes modernes ;
- cartes réutilisables ;
- Material Design 3 ;
- thème clair / sombre.

Contexte de l'application

Rappel : Votre application doit présenter le profil d'un développeur ou d'une développeuse.

Le profil peut être :

- votre propre profil ;
- un profil fictif ;
- un profil d'étudiant en recherche de stage ;
- un profil junior Android ;
- un profil full-stack ;
- un profil freelance.

L'application doit rester simple : elle affiche des **données locales** directement écrites dans le code Kotlin.

Ce que l'application doit contenir

L'écran doit présenter au minimum :

- ☑ le **nom** du développeur ;
- ☑ son **titre** ou **rôle** ;
- ☑ une courte **description** ;
- ☑ une **image** ou un avatar *placeholder* ;
- ☑ une liste de **compétences** ;
- ☑ une liste de **projets** ou **expériences** ;
- ☑ une zone de **contact** ;
- ☑ une **action** utilisateur, par exemple "Contacter", "Voir le portfolio" ou "Télécharger le CV".

Le clic sur le bouton ne doit pas forcément faire une vraie action. Il peut appeler un *callback* vide ou contenir un commentaire.



Notions à appliquer

1. Composants fondamentaux

L'application doit utiliser plusieurs composants Compose vus dans le bloc :

- **Text** pour les titres, descriptions et informations ;
- **Button** pour une action principale ;
- **Icon** pour accompagner une information ou une action ;
- **Image** pour l'avatar ou une illustration ;
- **Card** pour regrouper les informations ;
- **TextField** ou **OutlinedTextField** en **lecture seule** pour afficher une information structurée, par exemple email, disponibilité ou localisation.

2. Layouts Compose

L'écran doit être organisé avec :

- **Column** pour structurer l'écran **verticalement** ;
- **Row** pour aligner des informations sur une **même ligne** ;
- **Box** pour créer une **zone avec avatar, badge** ou élément superposé ;
- **Spacer** pour gérer les **espaces** ;
- **Surface** pour créer au moins une **zone visuelle** cohérente avec le **thème**.

3. Listes modernes

L'application doit afficher au moins une collection avec :

- **LazyColumn** ou **LazyRow**.

Exemples possibles :

- liste de **compétences** ;
- liste de **projets** ;
- liste **d'expériences** ;
- liste de **technologies** ;
- liste de **liens professionnels**.

Chaque élément de liste doit être affiché avec un composable réutilisable, par exemple :

- **SkillCard** ;
- **ProjectCard** ;
- **ExperienceItem** ;
- **TechnologyChip**.

4. Material Design 3

L'application doit avoir une identité visuelle cohérente.

Vous devez personnaliser ou utiliser correctement :

- **MaterialTheme.colorScheme** ;
- **MaterialTheme.typography** ;
- **MaterialTheme.shapes** ;
- un rendu **clair** ;
- un rendu **sombre**.

Il n'est pas demandé de créer un design complexe. L'objectif est d'avoir une interface lisible, cohérente et propre.



Ce qu'il ne faut pas faire

Pour rester dans le périmètre du cours, vous ne devez pas utiliser :

- ~~API distante ;~~
- ~~Retrofit ;~~
- ~~Room ;~~
- ~~ViewModel ;~~
- ~~StateFlow ;~~
- ~~Navigation Compose ;~~
- ~~vraie authentification ;~~
- ~~stockage local ;~~
- ~~gestion avancée de l'état ;~~
- ~~formulaire complet avec validation.~~

Les données doivent rester **locales**.

Livrables

Créez un **nouveau dépôt Git public** pour ce devoir.

Rendez un **dépôt Git public** contenant :

- le projet Android **complet** ;
- un fichier **README.md** ;
- au moins une **capture d'écran** ;
- au moins deux **previews** ;
- le **lien** du dépôt déposé dans le devoir Teams.

README attendu

Le **README.md** doit contenir :

1. le **titre** du projet ;
2. une courte **description** de l'application ;
3. la **liste des composants** Compose utilisés ;
4. la **liste des layouts** utilisés ;
5. l'**explication** de la liste affichée avec **LazyColumn** ou **LazyRow** ;
6. l'**explication** des choix de **thème** ;
7. une **capture d'écran** ;
8. une courte réponse à la question : « **Quelles notions du chapitre Compose UI avez-vous réutilisées dans cette application ?** »